

§ 11.2 蛮力算法

11.2.1 算法描述

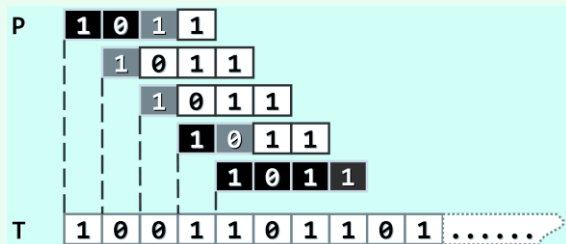


图11.1 串模式匹配的蛮力算法

蛮力串匹配是最直接最直觉的方法。如图11.1所示，可假想地将主串和模式串分别写在两条印有等间距方格的纸带上，主串对应的纸带固定，模式串纸带的首字符与主串纸带的首字符对齐，二者都沿水平方向放置。于是，只需将P与T中长度为m的 $n - m + 1$ 个子串逐一比对，即可确定可能的匹配位置。

不妨按自左向右的次序考查各子串。在初始状态下，T的前m个字符将与P的m个字符两两对齐。接下来，自左向右检查相互对齐的这m对字符：若当前字符对相互匹配，则转向下一对字符；反之一旦失配，则说明在此位置主串与模式串不可能完全匹配，于是可将P对应的纸带右移一个字符，然后从其首字符开始与T中对应的新子串重新对比。图中，模式串P的每一黑色方格对应于字符对的一次匹配，每一灰色方格对应于一次失配，白色方格则对应于未进行的一次比对。若经过检查，当前的m对字符均匹配，则意味着整体匹配成功，从而返回匹配子串的位置。

蛮力算法的正确性显而易见：既然只有在某一轮的m次比对全部成功之后才成功返回，故不致于误报；反过来，所有对齐位置都会逐一尝试，故亦不致漏报。

11.2.2 算法实现

以下给出蛮力算法的两个实现版本。二者原理相同、过程相仿，但分别便于引入后续的不同改进算法，故在此先做一比较。

```

1  /*****
2  * Text      :  0  1  2  .  .  .  i-j  .  .  .  i  .  .  n-1
3  *           -----|-----|-----
4  * Pattern   :                0  .  .  .  j  .  .
5  *           |-----|
6  *****/
7  int match(char* P, char* T) { //串匹配算法 (Brute-force-1)
8      size_t n = strlen(T), i = 0; //主串长度、当前接受比对字符的位置
9      size_t m = strlen(P), j = 0; //模式串长度、当前接受比对字符的位置
10     while (j < m && i < n) //自左向右逐个比对字符
11         if (T[i] == P[j]) //若匹配
12             { i++; j++; } //则转到下一对字符
13         else //否则
14             { i -= j - 1; j = 0; } //主串回退、模式串复位
15     return i - j; //如何通过返回值，判断匹配结果？
16 }

```

代码11.1 蛮力串匹配算法（版本一）

如代码11.1所示的版本借助整数*i*和*j*，分别指示*T*和*P*中当前接受比对的字符*T*[*i*]与*P*[*j*]。若当前字符对匹配，则*i*和*j*同时递增以指向下一对字符。一旦*j*增长到*m*则意味着发现了匹配，即可返回*P*相对于*T*的对齐位置*i* - *j*。一旦当前字符对失配，则*i*回退并指向*T*中当前对齐位置的下一字符，同时*j*复位至*P*的首字符处，然后开始下一轮比对。

```

1  /*****
2  * Text      :  0  1  2  .  .  .  i  i+1 .  .  .  i+j .  .  n-1
3  *
4  * Pattern   :                      0  1  .  .  .  j  .  .
5  *
6  *                      |-----|-----|
7  *****/
8  int match(char* P, char* T) { //串匹配算法 (Brute-force-2)
9      size_t n = strlen(T), i = 0; //主串长度、与模式串首字符的对齐位置
10     size_t m = strlen(P), j; //模式串长度、当前接受比对字符的位置
11     for (i = 0; i < n - m + 1; i++) { //主串从第i个字符起，与
12         for (j = 0; j < m; j++) //模式串中对应的字符逐个比对
13             if (T[i + j] != P[j]) break; //若失配，模式串整体右移一个字符，再做一轮比对
14         if (j >= m) break; //找到匹配子串
15     }
16     return i; //如何通过返回值，判断匹配结果？
17 }
```

代码11.2 蛮力串匹配算法（版本二）

如代码11.2所示的版本，借助整数*i*指示*P*相对于*T*的对齐位置，且随着*i*不断递增，对齐的位置逐步右移。在每一对齐位置*i*处，另一整数*j*从0递增至*m* - 1，依次指示当前接受比对的字符为*T*[*i* + *j*]与*P*[*j*]。因此，一旦发现匹配，即可直接返回当前的对齐位置*i*。

11.2.3 时间复杂度

从理论上讲，蛮力算法至多迭代*n* - *m* + 1轮，且各轮至多需进行*m*次比对，故总共只需做不超过(*n* - *m* + 1)·*m*次比对。那么，这种最坏情况的确会发生吗？答案是肯定的。

考查如图11.2所示的实例。无论采用上述哪个版本的蛮力算法，都需做*n* - *m* + 1轮迭代，且各轮都需做*m*次比对。因此，整个算法共需做*m*·(*n* - *m* + 1)次字符比对，其中成功的和失败的各有(*m* - 1)·(*n* - *m* + 1) + 1和*n* - *m* + 2次。因*m* << *n*，渐进的时间复杂度应为 $O(n \cdot m)$ 。



图11.2 蛮力算法的最坏情况

（也是基于坏字符策略BM算法的最好情况）



图11.3 蛮力算法的最好情况

（也是基于坏字符策略BM算法的最坏情况）

当然，蛮力算法的效率也并非总是如此低下。如图11.3所示，若将模式串*P*左右颠倒，则每经一次比对都可排除主串中的一个字符，故此类情况下的运行时间将为 $O(n)$ 。实际上，此类最好（或接近最好）情况出现的概率并不很低，尤其是在字符表较大时（习题[11-9]）。

§ 11.3 KMP算法

11.3.1 构思

上一节的分析表明，蛮力算法在最坏情况下所需时间，为主串长度与模式串长度的乘积，故无法应用于规模稍大的应用环境，很有必要改进。为此，不妨从分析以上最坏情况入手。

稍加观察不难发现，问题在于这里存在大量的局部匹配：每一轮的 m 次比对中，仅最后一次可能失配。而一旦发现失配，主串、模式串的字符指针都要回退，并从头开始下一轮尝试。

实际上，这类重复的字符比对操作没有必要。既然这些字符在前一轮迭代中已经接受过比对并且成功，我们也就掌握了它们的所有信息。那么，如何利用这些信息，提高匹配算法的效率呢？

以下以蛮力算法的前一版本（代码11.1）为基础进行改进。

■ 简单示例

如图11.4所示，用 $T[i]$ 和 $P[j]$ 分别表示当前正在接受比对的一对字符。

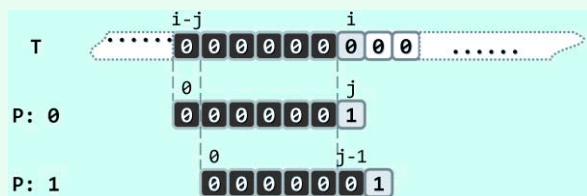


图11.4 利用以往的成功比对所提供的信息，可以避免主串字符指针的回退



图11.5 利用以往的成功比对所提供的信息，有可能使模式串大跨度地右移

当本轮比对进行到最后一对字符并发现失配后，蛮力算法会令两个字符指针同步回退（即令 $i = i - j + 1$ 和 $j = 0$ ），然后再从这一位置继续比对。然而事实上，指针 i 完全不必回退。

■ 记忆 = 经验 = 预知力

经过前一轮比对，我们已经清楚地知道，子串 $\text{substr}(T, i - j, j)$ 完全由'0'组成。记住这一性质便可预测出：在回退之后紧接着的下一轮比对中，前 $j - 1$ 次比对必然都会成功。因此，尽可令 i 保持不变、 $j = j - 1$ ，然后继续比对。如此，将使下一轮的比对减少 $j - 1$ 次！

上述“令 i 保持不变、 $j = j - 1$ ”的含义，可理解为“令 P 相对于 T 右移一个单元，然后从前一失配位置继续比对”。实际上这一技巧可推而广之：利用以往的成功比对所提供的信息（记忆），不仅可避免主串字符指针的回退，而且可使模式串尽可能大跨度地右移（经验）。

■ 一般实例

如图11.5所示，再来考查一个更具一般性的实例。

本轮比对进行到发现'E' = $T[i] \neq P[4] = 'O'$ 失配后，在保持 i 不变的同时，应将模式串 P 右移几个单元呢？有必要逐个单元地右移吗？不难看出，在这一情况下移动一个或两个单元都是徒劳的。事实上，根据此前的比对结果，必然有

$$\text{suffix}(\text{prefix}(T, i), 4) = \text{substr}(T, i - 4, 4) = \text{prefix}(P, 4) = \text{"REGR"}$$

若在此局部能够实现匹配，则至少紧邻于 $T[i]$ 左侧的若干字符均应得到匹配——比如，当 $P[0]$ 与 $T[i - 1]$ 对齐时，即属这种情况。注意到 $i - 1$ 是能够如此匹配的最左侧位置，故可放心地将 P 右移 $4 - 1 = 3$ 个单元（等效于 i 保持不变，同时令 $j = 1$ ），然后继续比对。